



Technical Documentation

Architecture, data model, API, security, deployment and operations

Product	ComplytEX — Safety, Compliance & Business Management (SaaS)
Intended readers	Developers, DevOps / IT administrators, technical reviewers
Application	https://safety.designerease.com
Document date	3 October 2026
Version	1.0 — reflects the code as deployed on this date

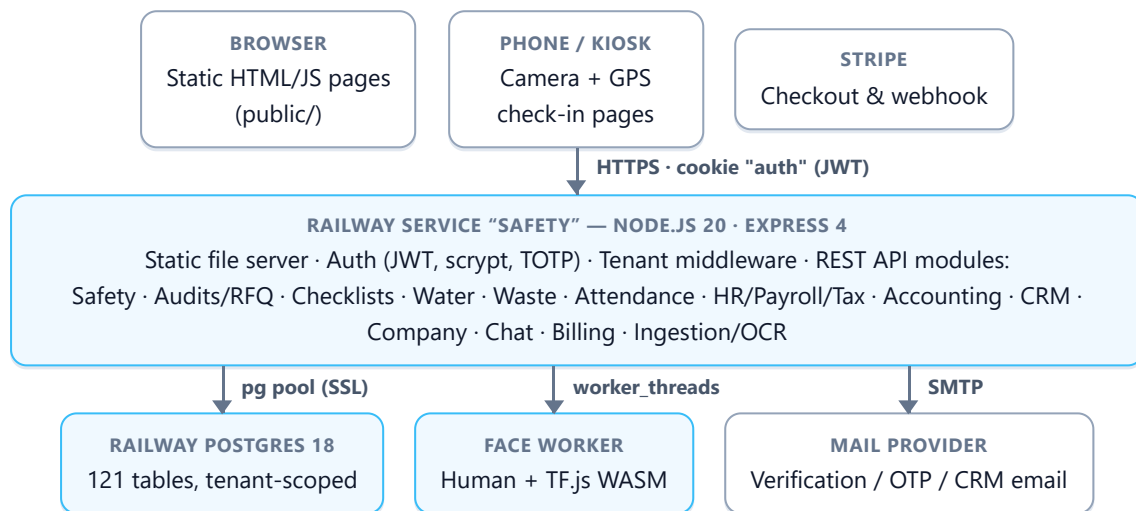
Contents

- 1. System overview**
 - 1.1 Architecture
 - 1.2 Technology stack
 - 1.3 Repository layout
- 2. Runtime and request pipeline**
 - 2.1 Start-up sequence
 - 2.2 Middleware order
 - 2.3 Feature flags
- 3. Configuration**
 - 3.1 Environment variables
- 4. Data model**
 - 4.1 Core entities
 - 4.2 Stored functions
 - 4.3 Migrations
 - 4.4 Table catalogue
- 5. Authentication and authorisation**
 - 5.1 Sessions
 - 5.2 Role-based landing
 - 5.3 Permission checks in code
- 6. Module internals**
 - 6.1 Safety checklists and audits
 - 6.2 Payroll and tax engine
 - 6.3 Salary advances
 - 6.4 Task planner
 - 6.5 Demo accounts switch
 - 6.6 Role-based header icons
 - 6.7 Face attendance
 - 6.8 Accounting integration points
 - 6.9 Assistants and ingestion
- 7. API reference**
- 8. Security**
 - 8.1 Controls in place
 - 8.2 Known gaps and recommendations
- 9. Deployment and operations**
 - 9.1 Railway
 - 9.2 Database changes
 - 9.3 Local development
 - 9.4 Monitoring and maintenance
- 10. Known limitations and roadmap**

1. System overview

ComplytEX is a multi-tenant SaaS web application: a single Node.js/Express server renders static HTML pages and exposes a JSON REST API backed by PostgreSQL. It is deployed on **Railway** with a Railway-managed **PostgreSQL** database. The application was migrated from Azure SQL Server; Postgres implementations of every active module are selected by feature flags (`ENABLE_SUPABASE_*`), and the SQL Server code paths remain in the source but are inactive.

1.1 Architecture



1.2 Technology stack

Layer	Technology
Runtime	Node.js ≥ 18 (Railway image: Node 20; development: Node 22)
Web framework	Express 4.18, cookie-parser, cors, multer (uploads)
Database	PostgreSQL (Railway, v18) via <code>pg</code> 8.16; SQL Server driver <code>mssql</code> retained for legacy paths
Authentication	<code>jsonwebtoken</code> (HS256, 7-day tokens), Node <code>crypto</code> . <code>script</code> password hashing, <code>speakeasy</code> TOTP 2FA, <code>qrcode</code>
Face recognition	<code>@vladmandic/human</code> 3.3.6 on <code>@tensorflow/tfjs</code> 4.22 + <code>tfjs-backend-wasm</code> , <code>jpeg-js</code>
Documents & OCR	<code>tesseract.js</code> (OCR), <code>pdf-parse</code> , <code>mammoth</code> (DOCX), <code>xlsx</code>
Payments	<code>stripe</code> 14 (Checkout, Billing Portal, webhooks)
Email	<code>nodemailer</code> 8 (SMTP)
Front end	Plain HTML/CSS/JavaScript (no build step); shared scripts in <code>public/js</code> ; Chart.js on dashboards
Hosting	Railway (Nixpacks builder), custom domain <code>safety.designerease.com</code>

1.3 Repository layout

Path	Contents
<code>src/web-server.js</code>	Express app: middleware, auth, core/CRM/HR/accounting/company/chat/billing routes, payroll engine, start-up
<code>src/safety-api-supabase.js</code>	Safety API (incidents, grievances, checklist forms, training, audits, RFQ/quotes/invoices, AI prompts, barcode attendance)
<code>src/attendance/</code>	<code>attendance-api.js</code> (face attendance + HR register), <code>face-engine.js</code> (worker client), <code>face-worker.js</code> (models)
<code>src/routes/*-supabase.js</code>	Water, waste and checklist APIs (Postgres)
<code>src/db/supabase-*.js</code>	Data-access modules: auth, core, CRM, HR, accounting, payroll tax; <code>supabase.js</code> creates the pg pool
<code>src/multi-agent.js</code> , <code>agents-config.js</code>	Rule-based in-app assistants and module catalogue
<code>src/safety-agent-api-supabase.js</code>	Document ingestion agent (upload → proposal → approve/reject)
<code>public/</code>	All pages: <code>masters/</code> role dashboards, <code>hr/</code> , <code>crm/</code> , <code>accounting/</code> , <code>attendance/</code> , <code>js/</code> shared scripts, <code>app.css</code>
<code>supabase/01-09_*.sql</code>	Database schema and migrations (Postgres)
<code>scripts/setup-supabase.js</code>	Applies the SQL files in order
<code>scripts/migrate-sqlserver-to-supabase.js</code>	One-off data migration from SQL Server
<code>railway.toml</code> , <code>nixpacks.toml</code>	Build and start configuration
<code>*.sql</code> in <code>db/</code> , <code>database/</code> ; <code>src/safety-api.js</code> ; <code>src/routes/*.js</code> (non-supabase)	Legacy SQL Server implementation — inactive while the flags are on

2. Runtime and request pipeline

2.1 Start-up sequence

1. `dotenv` loads environment variables; Express app and middleware are registered.
2. All routes are registered synchronously; in Postgres mode the safety, water, waste, checklist and attendance modules are mounted directly.
3. `app.listen(PORT)` starts immediately so Railway's health check (`GET /health`) passes quickly.
4. Background tasks: demo-tenant seeding (`AUTO_SEED_DEMO`), face-model warm-up in the worker thread, attendance photo-retention purge (after 60 s, then every 24 h).

2.2 Middleware order

#	Middleware	Purpose
1	<code>cors()</code>	Cross-origin requests (currently open to all origins)
2	JSON body parser	Up to 50 MB (camera frames, base64 attachments)
3	<code>cookie-parser</code>	Reads the <code>auth</code> cookie
4	<code>express.static(public)</code>	Serves pages and scripts (no auto index)
5	<code>/api</code> visitor logger	Fire-and-forget insert into <code>visitor_logs</code>
6	Route handlers	Per-route <code>requireAuth</code> where present
7	Tenant middleware on <code>/api/safety</code> , <code>/api/water</code> , <code>/api/waste</code> , <code>/api/attendance</code> , <code>/api/ai/prompts</code>	Requires login; overwrites any client-sent <code>tenantId</code> with the session's tenant; <code>/api/safety/self-check</code> stays public

2.3 Feature flags

Each `ENABLE_SUPABASE_*` flag (AUTH, CRM, HR, ACCOUNTING, SAFETY, CHAT, BILLING) switches its module from SQL Server to Postgres. **All must be 1 in production**; SQL Server is no longer available, so a disabled flag breaks that module.

3. Configuration

3.1 Environment variables

Variable	Required	Description
<code>SUPABASE_DB_URL</code>	Yes	Postgres connection string. Production: <code>\${Postgres.DATABASE_URL}</code> (private network). Local: the public proxy URL.
<code>ENABLE_SUPABASE_AUTH ... _BILLING</code>	Yes	Seven flags, all <code>1</code> .
<code>JWT_SECRET</code>	Yes	Long random secret for signing session tokens. Changing it signs everyone out.
<code>NODE_ENV</code>	Yes	<code>production</code> on Railway (enables the Secure cookie flag).
<code>PORT</code>	No	Set by Railway; local default 3000 (3003 used in development).
<code>APP_NAME</code>	No	Display name (ComplytEX).
<code>AUTO_SEED_DEMO</code>	No	<code>1</code> creates the Demo Factory tenant and demo accounts; set <code>0</code> in production if not wanted.
<code>SMTP_HOST/PORT/USER/PASS/FROM</code>	For email	Verification, OTP and CRM email.
<code>ENABLE_EMAIL_OTP</code> , <code>ENABLE_TOTP_2FA</code>	No	Second-factor options at login.
<code>RECAPTCHA_SITE_KEY/SECRET</code>	No	Registration captcha (skipped when unset).
<code>STRIPE_SECRET</code> , <code>STRIPE_WEBHOOK_SECRET</code> , <code>STRIPE_PRICE_BASIC/PRO/ENTERPRISE</code>	For billing	Subscriptions and credit top-ups.
<code>BILLING_SUCCESS_URL</code> , <code>BILLING_CANCEL_URL</code>	For billing	Stripe redirect URLs.
<code>LOG_LEVEL</code> , <code>MONITORING_INTERVAL_MINUTES</code> , <code>MAINTENANCE_INTERVAL_HOURS</code>	No	Agent loop and logging settings.
<code>AZURE_SQL_*</code> , <code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>	No (legacy)	Not used in Postgres mode; leave unset.

Files

`.env` (local) and `.env.railway` (paste into Railway → Safety → Variables → Raw Editor) are git-ignored. Never commit secrets.

4. Data model

All tables live in the `public` schema. Multi-tenancy is **row-based**: top-level business tables carry `tenant_id`; child tables (audit items, corrective actions, RFQ quotes/invoices, training attendees, tax rules) are scoped through their parent row. Migration 07 added `tenant_id` to the remaining top-level tables and made checklist-heading and water-invoice uniqueness per tenant.

4.1 Core entities

Entity	Notes
<code>tenants</code> , <code>business_types</code>	A company and its business type (Buyer, Manufacturer, Supplier, Designer, SafetyAuditor, SafetyOffice, Inspection, BuyingAgent).
<code>users</code> , <code>company_users</code>	Global user accounts (email unique, scrypt hash + salt as bytea) and their membership of tenants (title, owner flag, active).
<code>roles</code> , <code>permissions</code> , <code>role_permissions</code> , <code>user_roles</code> , <code>modules</code>	RBAC. Permission codes such as <code>MODULE_VIEW_HR</code> , <code>MODULE_MANAGE_HR</code> , <code>USER_MANAGE</code> , <code>TENANT_ADMIN</code> , <code>ATTENDANCE_KIOSK</code> . <code>fn_has_permission(tenant, user, code)</code> checks access.
<code>audit_logs</code> , <code>visitor_logs</code>	Business audit trail (registration, tax verification, face enrollment...) and API access log.

4.2 Stored functions

`02_functions.sql` provides Postgres equivalents of the former stored procedures, including `sp_register_owner` (tenant + owner + Owner role with all permissions), `sp_create_employee`, `sp_create_role`, `sp_grant_role_permissions`, `sp_assign_user_role` and `fn_has_permission`. Functions returning tables use `#variable_conflict use_column`.

4.3 Migrations

File	Content
<code>01_schema.sql</code>	Core, CRM, HR, accounting, chat/email tables
<code>02_functions.sql</code>	Stored functions
<code>03_seed.sql</code>	Business types, modules, permissions
<code>04_safety_schema.sql</code> , <code>05_safety_extra_schema.sql</code>	Safety, audits, RFQ, water, waste, checklist, attendance (barcode)
<code>06_payroll_tax_schema.sql</code>	Tax profiles, rules, payroll confirmations
<code>07_safety_tenant_scoping.sql</code>	<code>tenant_id</code> on 11 tables, per-tenant unique indexes
<code>08_attendance_face.sql</code>	Sites, settings, face enrollments, challenges, rejected attempts, evidence columns, kiosk permission

<code>09_business_types.sql</code>	Adds the BuyingAgent business type
<code>10_tasks_payroll_runs_advances.sql</code>	Per-company task planner, payroll-run guard (one run per month), salary advances and their deductions

All migrations are idempotent (`if not exists` , guarded inserts) and are applied with `npm run supabase:setup` or individually with a pg client.

4.4 Table catalogue

121 tables, generated from `supabase/01-10_*.sql` on 3 October 2026.

Core, tenants & security

Table	Columns	Defined in
<code>audit_logs</code>	<code>audit_id</code> , <code>tenant_id</code> , <code>user_id</code> , <code>action</code> , <code>entity</code> , <code>entity_id</code> , <code>details</code> , <code>created_at</code>	<code>01_schema.sql</code>
<code>business_types</code>	<code>business_type_id</code> , <code>code</code> , <code>name</code> , <code>created_at</code>	<code>01_schema.sql</code>
<code>company_users</code>	<code>company_user_id</code> , <code>tenant_id</code> , <code>user_id</code> , <code>title</code> , <code>is_owner</code> , <code>is_active</code> , <code>created_at</code>	<code>01_schema.sql</code>
<code>email_verifications</code>	<code>token_id</code> , <code>user_id</code> , <code>token</code> , <code>expires_at</code> , <code>used</code> , <code>created_at</code>	<code>01_schema.sql</code>
<code>modules</code>	<code>module_id</code> , <code>code</code> , <code>name</code> , <code>created_at</code>	<code>01_schema.sql</code>
<code>permissions</code>	<code>permission_id</code> , <code>code</code> , <code>description</code> , <code>module_id</code>	<code>01_schema.sql</code>
<code>role_permissions</code>	<code>role_id</code> , <code>permission_id</code>	<code>01_schema.sql</code>
<code>roles</code>	<code>role_id</code> , <code>tenant_id</code> , <code>name</code> , <code>is_system</code> , <code>created_at</code>	<code>01_schema.sql</code>
<code>tenants</code>	<code>tenant_id</code> , <code>name</code> , <code>business_type_id</code> , <code>external_id</code> , <code>is_active</code> , <code>created_at</code> , <code>updated_at</code>	<code>01_schema.sql</code>
<code>user_roles</code>	<code>tenant_id</code> , <code>user_id</code> , <code>role_id</code>	<code>01_schema.sql</code>
<code>user_two_factor</code>	<code>user_id</code> , <code>secret</code> , <code>enabled</code> , <code>created_at</code> , <code>last_used_at</code>	<code>01_schema.sql</code>
<code>users</code>	<code>user_id</code> , <code>email</code> , <code>full_name</code> , <code>password_hash</code> , <code>password_salt</code> , <code>is_active</code> , <code>email_verified</code> , <code>two_fa_enabled</code> , <code>two_fa_secret</code> , <code>cell_phone</code> , <code>designation</code> , <code>avatar_url</code> , <code>created_at</code> , <code>updated_at</code>	<code>01_schema.sql</code>
<code>visitor_logs</code>	<code>log_id</code> , <code>tenant_id</code> , <code>user_id</code> , <code>path</code> , <code>method</code> , <code>ip</code> , <code>user_agent</code> , <code>referrer</code> , <code>country</code> , <code>region</code> , <code>city</code> , <code>created_at</code>	<code>01_schema.sql</code>

Safety

Table	Columns	Defined in
boiler_safety	id, inspection_date, factory_id, location, inspected_by, safety_programme, factory_collaboration, boiler_design, boiler_inspection, inspection_verifies, rapid_depressurization, nameplate_visible, fire_outlets, low_water_indicator, maintenance_logbook, design_manuals, certified_engineers, safety_department, engineers_trained, advanced_training, practical_training, water_source, water_storage_tank, water_softener, boiler_pressure, water_level, temperature, chemical_used, chemical_type, dosage_rate, feed_method, p_h, alkalinity, hardness, chlorides, silica, dissolved_solids, blowdown_frequency, blowdown_method, steam_pressure, steam_temperature, steam_traps, insulation, condensate_return, condensate_tank, condensate_pump, ventilation, lighting, exhaust_fan, exhaust_duct, stack_temp_sensor, safety_valves, pressure_gauge, low_water_cutoff, flame_safeguard, emergency_shutoff, fuel_train_shutoff, interlock_overtemperature, interlock_low_water, interlock_flame_failure, relief_valve_pressure, observations, status, created_at, updated_at	04_safety_schema.sql
consultant_engagement	id, assessment_date, consultant_name, project_name, assessed_by, legal_registered, valid_licenses, international_memberships, no_false_claims, independent_verification, no_conflict_interest, c_vs_provided, key_personnel_identified, responsibilities_defined, scope_of_work_defined, deliverables_specified, timeline_realistic, payment_terms_clear, testing_independent, insurance_coverage, warranty_provided, comments, status, created_at, updated_at	04_safety_schema.sql
dsa	id, assessment_date, factory_id, location, assessed_by, bulk_drawings, site_inspection, visual_checks, missing_docs, ndt_testing, analytical_model, capacity_verified, load_paths, adequacy_checked, observations, status, created_at, updated_at	04_safety_schema.sql
electrical_safety	id, inspection_date, location, equipment_type, equipment_id, voltage, wiring_condition, grounding_status, circuit_breakers_working, electrical_panel_condition, emergency_shutoff_accessible, labeling_complete, deficiencies, corrective_actions, inspected_by, status, created_date, updated_date	04_safety_schema.sql

Table	Columns	Defined in
emergency_power	id, inspection_date, factory_id, location, inspected_by, site_plan, egress_layout, floor_plans, elevations, roof_plan, pump_room_fire_resistance, pump_room_openlined, pump_selection, pump_room_ventilation, drainage_sump, clearances, components_listed, standard_details, fire_doors, pipe_sizes, no_combustibles, structure_verified, live_loads, wind_loads, seismic_checked, dimensions_verified, visual_inspection, foundation_verified, structural_calculations, roof_design, fire_resistance, nonstructural_braced, floor_load_limits, equipment_foundations, anchoring, battery_fire_rooms, door_hung, light_working_space, penetrations_sealed, single_line_diagram, backup_power_sized, automatic_start, atsups_isolation, circuit_breakers, electrical_components_listed, cables_sized, earthing_grounding, surge_protection, emergency_lighting, supervisory_alarms, dedicated_battery_room, battery_ventilation, room_door_vents, battery_temperature, spill_containment, battery_racks, eyewash_station, smoke_heat_detectors, emergency_lighting_battery_room, clear_access, additional_notes, status, created_at, updated_at	04_safety_schema.sql
fire_safety	id, inspection_date, location, equipment_type, equipment_id, fire_extinguishers_count, fire_extinguishers_working, smoke_detectors_count, smoke_detectors_working, fire_alarms_working, emergency_exits_clear, fire_drill_conducted, last_drill_date, deficiencies, corrective_actions, inspected_by, status, created_date, updated_date	04_safety_schema.sql
gas_safety	id, inspection_date, factory_id, location, inspected_by, worker_safety, explosion_controls, prohibited_storage, nfpa_standard, technical_guidance, safe_storage, transitional_storage, transitional_compliance, room_enclosed, fire_doors, verified_compliant, ongoing_compliance, building_distance, exterior_walls, doors_rating, building_sole_use, ventilation_installed, building_approved, cylinders_stored_condition, cylinders_kept_away, cylinders_upright, cylinders_secured, valve_protection, full_empty_separation, empty_cylinders_marked, properly_sized, no_leaks, regularly_inspected, approved_connections, no_tools_valves, hydrostatic_testing, cylinders_checked, cylinders_moved_trucks, leaking_tagged, leaking_moved, warning_signs_posted, supplier_notified, inspections_protocols, ca_ps_prepared, reports_documented, observations, corrective_actions, status, created_at, updated_at	04_safety_schema.sql
health_hazards	id, assessment_date, location, hazard_type, chemical_exposure, chemical_names, noise_level, ventilation_adequate, ppe_available, ppe_types, first_aid_available, wash_stations_accessible, hazard_severity, risk_level, deficiencies, corrective_actions, assessed_by, status, created_date, updated_date	04_safety_schema.sql

Table	Columns	Defined in
safety_ai_prompt_templates	id, tenant_id, name, category, content, created_by, created_date, updated_date	05_safety_extra_schema.sql
safety_audit_corrective_action	id, audit_item_id, corrective_action_taken, attachments, corrective_action_by, corrective_action_on, created_date, resolution_mode, resolution_amount, status	04_safety_schema.sql
safety_audit_item	id, audit_id, head, value, no_issue, issue_details, attachments, suggested_action, informed_to, created_date, updated_date	04_safety_schema.sql
safety_audit_log	log_id, proposal_id, action, actor_id, message, created_at	04_safety_schema.sql
safety_audit_plan	id, tenant_id, audit_name, audit_type, audit_company, audit_date, status, created_by, created_date, updated_date	04_safety_schema.sql
safety_audit_reaudit	id, audit_id, notes, attachments, status, created_by, created_date	04_safety_schema.sql
safety_boiler_safety	id, tenant_id, form_data, created_by, created_date, updated_date	04_safety_schema.sql
safety_checklist_headings	id, tab_id, heading_text, heading_slug, display_order, created_at, created_by	04_safety_schema.sql
safety_checklist_items	id, tab_id, heading_text, heading_slug, item_text, options, display_order, created_at, created_by, updated_at, is_active	04_safety_schema.sql
safety_consultant	id, tenant_id, form_data, created_by, created_date, updated_date	04_safety_schema.sql
safety_documents	document_id, tenant_id, file_name, file_path, mime_type, size_bytes, extracted_text, uploaded_by, uploaded_at	04_safety_schema.sql
safety_dsa	id, tenant_id, form_data, created_by, created_date, updated_date	04_safety_schema.sql
safety_electrical	id, tenant_id, inspection_date, location, inspected_by, form_data, status, created_by, created_date, updated_date	04_safety_schema.sql
safety_emergency_power	id, tenant_id, form_data, created_by, created_date, updated_date	04_safety_schema.sql
safety_fire_safety	id, tenant_id, form_data, created_by, created_date, updated_date	04_safety_schema.sql
safety_gas_safety	id, tenant_id, form_data, created_by, created_date, updated_date	04_safety_schema.sql
safety_grievances	id, grievance_date, complainant_name, complainant_role, grievance_type, category, description, priority, assigned_to, resolution_details, status, created_date, updated_date	04_safety_schema.sql
safety_health_hazards	id, tenant_id, assessment_date, location, assessed_by, form_data, created_by, created_date, updated_date	04_safety_schema.sql
safety_incidents	id, incident_date, incident_type, location, department, description, injury_occurred, property_damage, severity, reported_by, investigation_status, corrective_action, status, created_date, updated_date	04_safety_schema.sql

Table	Columns	Defined in
safety_proposal_items	item_id, proposal_id, target_table, action, payload	04_safety_schema.sql
safety_proposals	proposal_id, tenant_id, title, description, status, document_id, submitted_by, submitted_at, approved_by, approved_at, rejected_by, rejected_at, rejection_reason	04_safety_schema.sql
safety_rfq	id, tenant_id, audit_id, audit_item_id, item_name, detail, qty, unit, needed_by, shipping_terms, payment_terms, other_terms, recipients, status, created_by, created_date, updated_date	04_safety_schema.sql
safety_rfq_invoice	id, quote_id, invoice_number, subtotal, tax, total, attachment_url, status, created_date, updated_date	04_safety_schema.sql
safety_rfq_quote	id, rfq_id, supplier_name, supplier_email, price, shipping_cost, tax, discount, total, arrival_date, shipping_terms, payment_terms, other_terms, status, approved, approved_date, po_number, created_date, updated_date	04_safety_schema.sql
safety_safety_training	id, tenant_id, form_data, created_by, created_date, updated_date	04_safety_schema.sql
safety_structural	id, tenant_id, inspection_date, location, structure_type, inspected_by, form_data, status, created_by, created_date, updated_date	04_safety_schema.sql
safety_training_attendee	id, session_id, employee_id, name, email, created_date	04_safety_schema.sql
safety_training_session	id, tenant_id, name, description, related_to, training_date, conducted_by, created_by, created_date, updated_date	04_safety_schema.sql
safety_ungp	id, tenant_id, form_data, created_by, created_date, updated_date	04_safety_schema.sql
safety_units	id, tenant_id, name, created_date	04_safety_schema.sql
safety_usc_safe	id, tenant_id, form_data, created_by, created_date, updated_date	04_safety_schema.sql
structural_safety	id, inspection_date, location, structure_type, floor_condition, wall_condition, ceiling_condition, staircase_condition, railings_secure, cracks_visible, water_leakage, load_bearing_capacity, deficiencies, corrective_actions, inspected_by, status, created_date, updated_date	04_safety_schema.sql

CRM

Table	Columns	Defined in
contact_test	id, name, created_date	04_safety_schema.sql
crm_accounts	id, tenant_id, name, created_at	01_schema.sql
crm_appointments	id, tenant_id, company, notes, when_at, created_by, created_at	01_schema.sql
crm_categories	id, tenant_id, role_code, name	01_schema.sql

Table	Columns	Defined in
crm_companies	id, tenant_id, company_name, company_details, company_type, product, hs_code, phone, cell_phone, whatsapp, website, address, city, state, zip, country, ntn, instagram, facebook, twitter, linkedin, skype, source_of_contact, status, saved_updated_on, send_email_on_save, created_by, created_at	01_schema.sql
crm_company_category_map	company_id, category_id	01_schema.sql
crm_contact_persons	id, company_id, first_name, last_name, designation, email, phone, cell_phone, whatsapp, linkedin, skype	01_schema.sql
crm_email_campaigns	id, tenant_id, name, subject, scheduled_at, created_by, created_at	01_schema.sql
crm_leads	id, tenant_id, lead_name, company, email, phone, status, estimated_value, source, notes, created_by, created_at, updated_at	01_schema.sql
crm_orders	id, tenant_id, order_number, order_type, party_name, contact_person, email, address, items, subtotal, tax, total_amount, status, notes, lead_id, created_by, created_at, updated_at, shipped_at	01_schema.sql
crm_segment_companies	segment_id, company_id, tenant_id	01_schema.sql

HR & payroll

Table	Columns	Defined in
hr_employee_pay_details	id, tenant_id, employee_id, salary_scale, pay_frequency, commission_code, fica_eobi, state_allowance, federal_withholding, city_withholding, country_filing_status, amount, deductions, blank_check_overtime_rate, ytd_regular_hours, last_agi, pregnancy_leaves, pay_yn, salary, commission_percent, federal_income_tax_yn, country_allowance, state_withholding, federal_filing_status, city_filing_status, net_amount, pre_tax_amount, ytd_fit, ytd_overtime_hours, last_fica_eobi, pay_type, hourly_rate, overtime_rate, federal_allowance, city_allowance, country_withholding_amount, state_filing_status, gender, blank_check_hourly_rate, additions, last_gross, last_fit, created_at, updated_at	01_schema.sql
hr_employee_profile	id, tenant_id, employee_id, active, phone, country, state, zip, fax, department, employee_type, ssn, email, hire_date, birth_date, commissionable, commission_percent, next_of_kin_name, next_of_kin_number, source_type, industry, geo_location, disability, work_injury, health_insurance, id_number, notes, heritage, sexual_orientation, job_description, created_at, updated_at	01_schema.sql
hr_employee_totals	id, tenant_id, employee_id, mtd_gross, mtd_state, qtd_gross, qtd_state, ytd_gross, ytd_state, mtd_fica, qtd_fica, ytd_fica, mtd_local, qtd_local, ytd_local, mtd_federal, qtd_federal, ytd_federal, mtd_other, qtd_other, ytd_other, updated_at	01_schema.sql

Table	Columns	Defined in
hr_payroll_run	id, tenant_id, period, generated_by, generated_at, employee_count, total_net, total_advance_deductions	10_tasks_payroll_runs_advances.sql
hr_payroll_tax_confirmation	id, tenant_id, period, profile_id, confirmed_by, confirmed_at	06_payroll_tax_schema.sql
hr_salary_advance	id, tenant_id, employee_id, amount, installments, monthly_deduction, start_period, reason, status, repaid_amount, requested_by, requested_at, decided_by, decided_at, decision_note, ap_invoice_order_id, updated_at	10_tasks_payroll_runs_advances.sql
hr_salary_advance_deduction	id, advance_id, tenant_id, period, amount, payroll_run_id, created_at	10_tasks_payroll_runs_advances.sql
hr_tax_profile	id, tenant_id, name, country, currency, effective_from, effective_to, is_active, source_reference, notes, verified_by, verified_at, created_by, created_at, updated_at	06_payroll_tax_schema.sql
hr_tax_rule	id, profile_id, sort_order, name, category, payer, method, rate, base_cap, amount_cap, fixed_amount, slabs, created_at	06_payroll_tax_schema.sql

Accounting & billing

Table	Columns	Defined in
ap_invoices	id, tenant_id, order_id, product_name, due_date, supplier_name, amount, status, doc_url, notes, bank_id, created_by, created_at, approved_by, approved_at, paid_by, paid_at	01_schema.sql
ar_invoices	id, tenant_id, order_id, product_name, due_date, customer_name, amount, status, doc_url, notes, bank_id, created_by, created_at, received_by, received_at	01_schema.sql
bank_ledger	ledger_id, tenant_id, bank_id, entry_type, amount, quantity, reference, party, slip_number, doc_url, entry_date, created_by	01_schema.sql
banks	bank_id, tenant_id, name, account_title, account_number, branch_name, notes, opening_balance, created_at	01_schema.sql
chart_of_accounts	id, tenant_id, account_type, account_name, account_number, notes, created_at, updated_at	01_schema.sql
credit_top_ups	top_up_id, tenant_id, stripe_session_id, stripe_payment_intent, credits, amount_cents, currency, status, created_at	01_schema.sql
credit_usage	usage_id, tenant_id, user_id, credits, reason, created_at	01_schema.sql
subscriptions	subscription_id, tenant_id, stripe_customer_id, stripe_subscription_id, status, plan_name, credit_balance, current_period_end, cancel_at_period_end, created_at	01_schema.sql

Attendance

Table	Columns	Defined in
attendance_challenge	id, tenant_id, issued_to, mode, action, created_at, expires_at, used_at	08_attendance_face.sql

Table	Columns	Defined in
attendance_face_enrollment	id, tenant_id, user_id, embedding, photo, quality, consent_at, consent_by, enrolled_by, active, created_at	08_attendance_face.sql
attendance_rejected_attempt	id, tenant_id, claimed_user_id, matched_user_id, recorded_by, mode, reasons, match_score, liveness_score, latitude, longitude, gps_accuracy_m, ip, user_agent, photo, created_at	08_attendance_face.sql
attendance_settings	tenant_id, match_threshold, review_threshold, liveness_threshold, require_geofence, max_gps_accuracy_m, min_minutes_between, photo_retention_days, updated_at	08_attendance_face.sql
attendance_site	id, tenant_id, name, latitude, longitude, radius_m, active, created_at, updated_at	08_attendance_face.sql
safety_attendance_barcode	id, tenant_id, user_id, barcode, photo_url, created_date	05_safety_extra_schema.sql
safety_attendance_log	id, tenant_id, user_id, direction, timestamp, source, device_id, notes, method, match_score, liveness_score, latitude, longitude, gps_accuracy_m, site_id, distance_m, ip, user_agent, photo, flags, review_status, reviewed_by, reviewed_at, review_note, recorded_by	05_safety_extra_schema.sql

Water & waste

Table	Columns	Defined in
waste_management	id, date_of_waste_generation, waste_category, type_of_waste, quantity_of_waste, unit, source_of_waste, collection_method, storage_method, storage_duration_days, warehouse_name, location, notes, created_at, updated_at, created_by, updated_by	04_safety_schema.sql
water_buying	id, supplier_name, supplier_contact, date_of_purchase, quantity_purchased, unit, cost_per_unit, total_cost, payment_method, invoice_number, water_tank, notes, created_at, updated_at, created_by, updated_by	04_safety_schema.sql
water_discharge_quality	id, monitoring_frequency, sampling_points_locations, person_responsible, comments_observations, laboratory_used, parameters_monitored, result_value, units, compliance_standards, monitoring_equipment, monitoring_date, created_at, updated_at, created_by, updated_by	04_safety_schema.sql
water_rain_collection	id, collection_date, collection_location, quantity_collected, collection_unit, rain_tank, water_quality, treatment_required, treatment_type, collected_by, notes, created_at, updated_at, created_by, updated_by	04_safety_schema.sql
water_recycling	id, department, date_of_recycling, recycling_method, quantity_recycled, unit, water_tank, notes, created_at, updated_at, created_by, updated_by	04_safety_schema.sql
water_usage	id, department, date_of_usage, quantity_used, unit, purpose_of_usage, water_efficient_tech, reduction_percentage, source_of_water, available_qty, usage_month, notes, created_at, updated_at, created_by, updated_by	04_safety_schema.sql

Table	Columns	Defined in
water_waste	id, department, date_of_waste_generation, quantity_of_wastewater, unit, type_of_wastewater, wastewater_treatment_process, percentage_of_pollutant_removal, disposal_method, quantity_disposed, disposed_unit, notes, created_at, updated_at, created_by, updated_by	04_safety_schema.sql

Other (company setup, chat, tasks, email)

Table	Columns	Defined in
candidate	id, tenant_id, job_posting_id, name, email, phone, address, linkedin_url, work_auth_status, preferred_start_date, status, applied_at, shortlisted, shortlisted_at	01_schema.sql
candidate_evaluation	id, candidate_id, tenant_id, evaluator_name, technical_skills, teamwork, leadership, problem_solving, communication, test_details, agenda, recommendation, created_at	01_schema.sql
candidate_interview	id, candidate_id, tenant_id, interview_date, interview_time_slot, interview_type, interviewer_name, interviewer_title, reason, created_at	01_schema.sql
candidate_offer	id, candidate_id, tenant_id, employment_type, benefits, salary_offer, start_date, confidentiality, non_compete, at_will, offer_status, created_at	01_schema.sql
certifications	certification_id, tenant_id, name, issuer, issue_date, expiry_date, created_at	01_schema.sql
company_assets	id, tenant_id, asset_name, asset_type, quantity, purchase_date, purchase_price, supplier, notes, saved_on, created_at, updated_at	01_schema.sql
company_certification	id, tenant_id, type, policy, name, valid_from, valid_till, detail, cert_image_url, logo_url, created_at, updated_at	01_schema.sql
company_product_line	id, tenant_id, name, created_at	01_schema.sql
company_profile	tenant_id, city, state, postal_code, country, employees, capacity_type, capacity_qty, min_order_qty, sales_profit_type, sales_profit_value, min_wage, min_age, retirement_age, deal_in_garments, deal_in_home_textile, logo_url, updated_at	01_schema.sql
company_space	id, tenant_id, name, type, purpose, floor, width, length, height, unit, capacity, ventilation, doors, windows, exit_plan_url, created_at, updated_at	01_schema.sql
company_union	id, tenant_id, name, description, created_at, updated_at	01_schema.sql
company_warehouse	id, tenant_id, name, address_type, warehouse_type, city, state, postal_code, country, created_at, updated_at	01_schema.sql

Table	Columns	Defined in
conversation_members	id, conversation_id, user_id, added_at	01_schema.sql
conversations	id, tenant_id, title, created_by, created_at	01_schema.sql
customers	customer_id, tenant_id, name, email, phone, created_at	01_schema.sql
emails	id, tenant_id, user_id, folder, from_address, to_addresses, cc_addresses, subject, body, attachments_json, sent_at, created_at	01_schema.sql
invoices	invoice_id, tenant_id, stripe_invoice_id, amount_cents, currency, status, created_at	01_schema.sql
job_posting	id, tenant_id, title, department, type, openings, location, salary_range, description, required_qualification, desired_qualification, application_deadline, diversity_flag, eoe_flag, posted_at	01_schema.sql
login_otps	id, user_id, code, expires_at, consumed, created_at	01_schema.sql
messages	id, conversation_id, user_id, body, created_at	01_schema.sql
phone_notes	id, tenant_id, subject, body, follow_up_at, customer_name, sales_person_name, created_by, created_at	01_schema.sql
planner_task	id, tenant_id, title, task_date, assignee_id, start_hour, end_hour, priority, note, created_by, created_at, updated_at	10_tasks_payroll_runs_advances.sql
segments	id, tenant_id, name, created_at	01_schema.sql
suppliers	supplier_id, tenant_id, name, email, phone, created_at	01_schema.sql
ticket_messages	message_id, ticket_id, user_id, message, created_at	01_schema.sql
tickets	ticket_id, tenant_id, created_by, subject, status, priority, created_at	01_schema.sql

5. Authentication and authorisation

5.1 Sessions

- **Login** (`POST /api/auth/login`) verifies the script hash and issues a JWT `{uid, tid}` signed with `JWT_SECRET`, valid 7 days, stored in cookie `auth` (`HttpOnly`, `SameSite=Lax`, `Secure` in production, `Max-Age` 7 days).
- `requireAuth` verifies the cookie and sets `req.auth`. The tenant middleware does the same for the company-scoped API prefixes.
- Front end: `master-layout.js` calls `/api/auth/me`; a 401 redirects to `/login?next=...` (public pages exempt). The login page honours `next` for same-site paths only.
- Optional second factors: TOTP (speakeasy, QR enrolment) or email OTP.
- Registration normalises the form's business type (e.g. `vendor` → `Supplier`), verifies reCAPTCHA if configured, calls `sp_register_owner` and sends a verification email.

5.2 Role-based landing

`determineRedirect()` maps role names to start pages: Safety Officer/Office → `/masters/safety-office.html`; Safety Auditor → `/masters/safety-auditor.html`; Inspector/Inspection, Buyer, Supplier, Designer → their master pages; otherwise `/dashboard`.

5.3 Permission checks in code

Action	Rule
Create/edit/verify/delete tax profiles	<code>MODULE_MANAGE_HR</code>
Face enrollment, attendance settings/sites, register, review	<code>MODULE_MANAGE_HR</code>
Kiosk check-in (1:N identification)	<code>ATTENDANCE_KIOSK</code> or <code>MODULE_MANAGE_HR</code>
Set corrective-action status	Role name "Safety Auditor" or "Audit Officer" (otherwise status is ignored)
All company-scoped safety/water/waste/attendance data	Signed-in user's tenant only; other tenants' ids return 404

6. Module internals

6.1 Safety checklists and audits

- Each checklist (fire, electrical, structural, health, usc-safe, gas, boiler, consultant, dsa, emergency-power, training, ungp) is stored as **one JSON row per tenant** (`form_data`) in `safety_<name>` , upserted on save.
- Custom questions/headings live in `safety_checklist_headings/items` per tenant and tab.
- Planning an audit snapshots the current checklist answers into `safety_audit_item` rows (`NoIssue=null` → status *Planned*).
- Saving the auditor sheet **updates items in place by id** (inserts new, deletes removed) so corrective actions stay linked. Plan status: no answers → `Planned` ; issues → `OpenIssues` (or `Closed` if all issues are closed); none → `Closed` .
- Corrective actions are append-only history in `safety_audit_corrective_action` ; the latest row per item is current. Inserting one sets the plan to `CorrectiveSubmitted` , or `Closed` when every issue item's latest status is `Closed` .
- RFQ chain: `safety_rfq` → `safety_rfq_quote` (approve sets PO number) → `safety_rfq_invoice` .

6.2 Payroll and tax engine

- Monthly basic = MTD gross, else salary adjusted for frequency (semi-monthly ÷2, bi-weekly ×12/26, weekly ×12/52).
- The active profile is the tenant's `is_active` profile whose effective dates cover the first day of the period (latest `effective_from` wins).
- Rule methods: **annual_slabs** — $\text{tax}(\text{annual} = \text{gross} \times 12) = \text{fixed} + \text{rate\%} \times (\text{annual} - \text{from})$ of the matching slab, ÷12; **percent** — $\text{rate\%} \times \min(\text{gross}, \text{base cap})$, capped at amount cap; **fixed** — flat monthly amount. Employee flags `federal_income_tax_yn=false` / `fica_eobi=false` skip income-tax / social-security rules.
- Manual amounts in Totals/Pay Details override the profile for that employee (`TaxSource='manual'`). There is no default rate.
- `POST /api/hr/payroll/generate` requires a verified profile for the period and `confirmTaxes: true` ; records `hr_payroll_tax_confirmation` ; creates one AP invoice per employee with net pay > 0. Editing a profile clears `verified_at` .
- One run per month:** generation first claims `hr_payroll_run(tenant_id, period)` (unique); a second attempt returns 409 `PAYROLL_ALREADY_GENERATED` . Months generated before this table existed are detected from `PAY-<period>-*` payables. If a run fails before posting anything, the claim is released.

6.3 Salary advances

- `hr_salary_advance` : amount, installments (1–24), monthly deduction (= amount ÷ installments), first deduction period, status *Pending* → *Approved* → *Repaid* (or *Rejected* / *Cancelled*).
- Employees request their own; `MODULE_MANAGE_HR` users request for anyone and decide. The requester (or the employee concerned) cannot decide their own advance. Rejection requires a note.
- Approval (in one transaction) creates AP invoice `ADV-<id>` for the payout.
- Payroll deducts, per approved advance whose start period has arrived, `min(monthly deduction, outstanding)` , capped at the employee's pay after tax and deductions; deductions are stored in

`hr_salary_advance_deduction` (unique per advance and period) and update `repaid_amount` .

6.4 Task planner

Tasks are rows in `planner_task` scoped by `tenant_id` (previously a JSON file shared by all companies and lost on each deploy). The API shape (`id`, `title`, `date`, `assigneeId`, `startHour`, `endHour`, `priority`, `note`) is unchanged.

6.5 Demo accounts switch

`AUTO_SEED_DEMO=0` stops demo seeding, makes `/api/public-config` report `demoAccounts:false` (the login page then hides the demo buttons) and refuses logins for `...@demo.example` . Demo data is not deleted, so the switch is reversible.

6.6 Role-based header icons

`master-layout.js` hides module icons not relevant to the user's role names (Owner/admin: all; unrecognised roles: all). This is a convenience only — permissions are enforced by the server.

6.7 Face attendance

- **Engine:** `face-worker.js` runs Human (blazeface detector, facemesh, iris, faceres 1024-d embedding, antispoof, liveness) on the TF.js WASM backend inside a `worker_thread` ; jobs are serialised. Model and WASM files are read from `node_modules` via a worker-local `fetch` shim for `file://` URLs.
- **Enrollment** (HR, consent required): 3–5 JPEG frames; each must contain one face $\geq 6\%$ of the frame, $|\text{yaw}| \leq 15^\circ$, antispoof $\geq$ half the threshold; frames must match each other (≥ 0.60); the averaged embedding must not match another enrolled employee ($\geq$ match threshold).
- **Challenge:** `POST /api/attendance/challenge` issues a one-time UUID with action `turn_left|turn_right` , valid 120 s, max 15 per user per 10 minutes. It is claimed atomically on use.
- **Check-in:** 2–4 frames (neutral, turned..., neutral). Refused for: no/multiple faces, face $< 3\%$ of frame, frames not the same person (< 0.55), head-turn delta < 0.10 eye-widths or wrong direction, antispoof $< \frac{1}{2}$ threshold, identity below match threshold (self: 1:1; kiosk: 1:N), missing location or outside geofence (when required), duplicate within N minutes. Flagged for: weak match, low liveness, kiosk margin < 0.06 , poor GPS, no site configured, impossible travel (> 200 km/h over > 2 km), new device.
- Accepted entries go to `safety_attendance_log` (method `face` , scores, coordinates, site, distance, IP, user agent, 240 px evidence photo, flags, `review_status`); refusals go to `attendance_rejected_attempt` . Server time is authoritative.
- **Retention:** photos older than `photo_retention_days` (default 90) are removed from non-pending rows daily; expired challenges are deleted.

6.8 Accounting integration points

AP invoices are created by: manual entry, payroll generation, CRM procurement orders (`/create-ap`) and asset purchases with a price. AR invoices come from manual entry and CRM sales orders (`/create-ar`). Paying AP or receiving AR updates bank balances and the ledger.

6.9 Assistants and ingestion

The in-app “agents” (`multi-agent.js`) are rule-based; no external LLM provider is configured. “AI Suggest” uses built-in checklist suggestion lists. The Safety Agent extracts text from uploads (PDF/TXT) into a pending proposal that changes data only after approval. CRM business-card upload uses Tesseract OCR.

7. API reference

All endpoints return JSON `{ success: boolean, data?, error? }` (checklist heading/item endpoints return plain arrays). Errors use HTTP 400 (validation), 401 (not signed in), 403 (permission), 404 (not found or another company's record), 409 (conflict), 422 (attendance refusal with `reasons[ ]`), 429 (rate limit) and 500.

292 endpoints, generated from the source code on 3 October 2026. **Login** = requires a signed-in user (cookie `auth`); company-scoped routes use the signed-in user's company only. **Admin** = signed-in user with `USER_MANAGE`. **Public*** = no login check in code (see Security notes).

Method	Path	Access	Source
Health			
GET	/health	Public	web-server.js
accounting			
GET	/api/accounting/accounts	Login	web-server.js
POST	/api/accounting/accounts	Login	web-server.js
DELETE	/api/accounting/accounts/:id	Login	web-server.js
GET	/api/accounting/ap	Login	web-server.js
POST	/api/accounting/ap	Login	web-server.js
POST	/api/accounting/ap/:id/approve	Login	web-server.js
POST	/api/accounting/ap/:id/pay	Login	web-server.js
GET	/api/accounting/ar	Login	web-server.js
POST	/api/accounting/ar	Login	web-server.js
POST	/api/accounting/ar/:id/receive	Login	web-server.js
GET	/api/accounting/bank-summary	Login	web-server.js
GET	/api/accounting/banks	Login	web-server.js
POST	/api/accounting/banks	Login	web-server.js
GET	/api/accounting/banks/:id	Login	web-server.js
POST	/api/accounting/deposits	Login	web-server.js
GET	/api/accounting/ledger	Login	web-server.js
POST	/api/accounting/ledger	Login	web-server.js
POST	/api/accounting/transfer	Login	web-server.js
agent			
GET	/api/agent/status	Public	web-server.js
agents			
POST	/api/agents/chat	Login	web-server.js
GET	/api/agents/modules	Login	web-server.js
ai			
GET	/api/ai/prompts	Login	safety-api-supabase.js
POST	/api/ai/prompts	Login	safety-api-supabase.js

Method	Path	Access	Source
DELETE	/api/ai/prompts/:id	Login	safety-api-supabase.js
GET	/api/ai/prompts/:id	Login	safety-api-supabase.js
PUT	/api/ai/prompts/:id	Login	safety-api-supabase.js
attendance			
GET	/api/attendance/barcodes	Login	safety-api-supabase.js
POST	/api/attendance/barcodes	Login	safety-api-supabase.js
DELETE	/api/attendance/barcodes/:id	Login	safety-api-supabase.js
POST	/api/attendance/challenge	Login	attendance/attendance-api.js
GET	/api/attendance/enrollments	Login	attendance/attendance-api.js
POST	/api/attendance/enrollments	Login	attendance/attendance-api.js
DELETE	/api/attendance/enrollments/:userId	Login	attendance/attendance-api.js
POST	/api/attendance/face-checkin	Login	attendance/attendance-api.js
GET	/api/attendance/logs	Login	safety-api-supabase.js
POST	/api/attendance/logs	Login	safety-api-supabase.js
POST	/api/attendance/logs/:id/review	Login	attendance/attendance-api.js
GET	/api/attendance/me	Login	attendance/attendance-api.js
GET	/api/attendance/register	Login	attendance/attendance-api.js
GET	/api/attendance/rejected	Login	attendance/attendance-api.js
GET	/api/attendance/review	Login	attendance/attendance-api.js
GET	/api/attendance/settings	Login	attendance/attendance-api.js
PUT	/api/attendance/settings	Login	attendance/attendance-api.js
POST	/api/attendance/sites	Login	attendance/attendance-api.js
DELETE	/api/attendance/sites/:id	Login	attendance/attendance-api.js
GET	/api/attendance/status	Login	safety-api-supabase.js
auth			
POST	/api/auth/2fa/disable	Login	web-server.js
POST	/api/auth/2fa/enable	Login	web-server.js
POST	/api/auth/2fa/setup	Login	web-server.js
GET	/api/auth/2fa/status	Login	web-server.js
POST	/api/auth/login	Public	web-server.js
POST	/api/auth/logout	Public	web-server.js
POST	/api/auth/manual-verify	Admin	web-server.js
GET	/api/auth/me	Login	web-server.js
POST	/api/auth/register	Public	web-server.js
GET	/api/auth/session	Public	web-server.js
POST	/api/auth/switch-tenant	Login	web-server.js

Method	Path	Access	Source
GET	/api/auth/tenants	Login	web-server.js
POST	/api/auth/test-smtp	Admin	web-server.js
POST	/api/auth/verify-2fa	Public	web-server.js
GET	/api/auth/verify-email	Public	web-server.js
billing			
GET	/api/billing/invoices	Login	web-server.js
POST	/api/billing/portal	Login	web-server.js
GET	/api/billing/status	Login	web-server.js
POST	/api/billing/subscribe	Login	web-server.js
POST	/api/billing/topup	Login	web-server.js
POST	/api/billing/webhook	Public	web-server.js
chat			
GET	/api/chat/conversations	Login	web-server.js
POST	/api/chat/conversations	Login	web-server.js
GET	/api/chat/conversations/:id/messages	Login	web-server.js
POST	/api/chat/conversations/:id/messages	Login	web-server.js
POST	/api/chat/conversations/create	Login	web-server.js
POST	/api/chat/create	Login	web-server.js
POST	/api/chat/start	Login	web-server.js
GET	/api/chat/users	Login	web-server.js
company			
GET	/api/company/assets	Login	web-server.js
POST	/api/company/assets	Login	web-server.js
DELETE	/api/company/assets/:id	Login	web-server.js
PUT	/api/company/assets/:id	Login	web-server.js
GET	/api/company/certifications	Login	web-server.js
POST	/api/company/certifications	Login	web-server.js
DELETE	/api/company/certifications/:id	Login	web-server.js
POST	/api/company/certifications/:id/image	Login	web-server.js
POST	/api/company/certifications/:id/logo	Login	web-server.js
POST	/api/company/logo	Login	web-server.js
GET	/api/company/product-lines	Login	web-server.js
POST	/api/company/product-lines	Login	web-server.js
DELETE	/api/company/product-lines/:id	Login	web-server.js
GET	/api/company/profile	Login	web-server.js
PUT	/api/company/profile	Login	web-server.js

Method	Path	Access	Source
GET	/api/company/spaces	Login	web-server.js
POST	/api/company/spaces	Login	web-server.js
DELETE	/api/company/spaces/:id	Login	web-server.js
POST	/api/company/spaces/:id/exit-plan	Login	web-server.js
GET	/api/company/unions	Login	web-server.js
POST	/api/company/unions	Login	web-server.js
DELETE	/api/company/unions/:id	Login	web-server.js
PUT	/api/company/unions/:id	Login	web-server.js
GET	/api/company/warehouses	Login	web-server.js
POST	/api/company/warehouses	Login	web-server.js
DELETE	/api/company/warehouses/:id	Login	web-server.js
PUT	/api/company/warehouses/:id	Login	web-server.js
contacts			
DELETE	/api/contacts	Login	web-server.js
GET	/api/contacts	Login	web-server.js
POST	/api/contacts	Login	web-server.js
DELETE	/api/contacts/:id	Login	web-server.js
PUT	/api/contacts/:id	Login	web-server.js
crm			
GET	/api/crm/appointments	Login	web-server.js
POST	/api/crm/appointments	Login	web-server.js
GET	/api/crm/campaigns	Login	web-server.js
POST	/api/crm/campaigns	Login	web-server.js
GET	/api/crm/categories	Login	web-server.js
GET	/api/crm/companies	Login	web-server.js
POST	/api/crm/companies	Login	web-server.js
DELETE	/api/crm/companies/:id	Login	web-server.js
GET	/api/crm/companies/:id	Login	web-server.js
PUT	/api/crm/companies/:id	Login	web-server.js
POST	/api/crm/companies/:id/upgrade	Login	web-server.js
POST	/api/crm/contacts/ingest-card	Login	web-server.js
GET	/api/crm/countries	Login	web-server.js
GET	/api/crm/email	Login	web-server.js
POST	/api/crm/email	Login	web-server.js
DELETE	/api/crm/email/:id	Login	web-server.js
GET	/api/crm/email/:id	Login	web-server.js

Method	Path	Access	Source
GET	/api/crm/leads	Login	web-server.js
POST	/api/crm/leads	Login	web-server.js
PUT	/api/crm/leads/:id	Login	web-server.js
GET	/api/crm/notes	Login	web-server.js
POST	/api/crm/notes	Login	web-server.js
GET	/api/crm/orders	Login	web-server.js
POST	/api/crm/orders	Login	web-server.js
PUT	/api/crm/orders/:id	Login	web-server.js
POST	/api/crm/orders/:id/create-ap	Login	web-server.js
POST	/api/crm/orders/:id/create-ar	Login	web-server.js
POST	/api/crm/orders/:id/ship	Login	web-server.js
GET	/api/crm/segments	Login	web-server.js
POST	/api/crm/segments	Login	web-server.js
DELETE	/api/crm/segments/:id	Login	web-server.js
GET	/api/crm/segments/:id/companies	Login	web-server.js
POST	/api/crm/segments/:id/companies	Login	web-server.js
GET	/api/crm/summary	Login	web-server.js
demo			
POST	/api/demo/seed-employees	Admin	web-server.js
hr			
GET	/api/hr/advances	Login	web-server.js
POST	/api/hr/advances	Login	web-server.js
POST	/api/hr/advances/:id/cancel	Login	web-server.js
POST	/api/hr/advances/:id/decision	Login	web-server.js
GET	/api/hr/applicants	Login	web-server.js
POST	/api/hr/applicants	Login	web-server.js
POST	/api/hr/applicants/:id/evaluation	Login	web-server.js
POST	/api/hr/applicants/:id/offer	Login	web-server.js
POST	/api/hr/applicants/:id/shortlist	Login	web-server.js
GET	/api/hr/employees	Login	web-server.js
POST	/api/hr/employees	Login	web-server.js
GET	/api/hr/employees/:id	Login	web-server.js
PUT	/api/hr/employees/:id	Login	web-server.js
GET	/api/hr/employees/:id/pay-details	Login	web-server.js
PUT	/api/hr/employees/:id/pay-details	Login	web-server.js
GET	/api/hr/employees/:id/totals	Login	web-server.js

Method	Path	Access	Source
PUT	/api/hr/employees/:id/totals	Login	web-server.js
POST	/api/hr/employees/full	Login	web-server.js
POST	/api/hr/payroll/generate	Login	web-server.js
GET	/api/hr/payroll/preview	Login	web-server.js
POST	/api/hr/payroll/seed-salary	Login	web-server.js
GET	/api/hr/postings	Login	web-server.js
POST	/api/hr/postings	Login	web-server.js
DELETE	/api/hr/postings/:id	Login	web-server.js
PUT	/api/hr/postings/:id	Login	web-server.js
GET	/api/hr/tax-profiles	Login	web-server.js
POST	/api/hr/tax-profiles	Login	web-server.js
DELETE	/api/hr/tax-profiles/:id	Login	web-server.js
PUT	/api/hr/tax-profiles/:id	Login	web-server.js
POST	/api/hr/tax-profiles/:id/verify	Login	web-server.js
POST	/api/hr/tax-profiles/calculate	Login	web-server.js
GET	/api/hr/tax-status	Login	web-server.js
ingest			
POST	/api/ingest/docx	Login	web-server.js
POST	/api/ingest/excel	Login	web-server.js
POST	/api/ingest/ocr	Login	web-server.js
POST	/api/ingest/pdf	Login	web-server.js
permissions			
GET	/api/permissions	Login	web-server.js
profile			
GET	/api/profile	Login	web-server.js
PUT	/api/profile	Login	web-server.js
POST	/api/profile/avatar	Login	web-server.js
POST	/api/profile/change-password	Login	web-server.js
public-config			
GET	/api/public-config	Public	web-server.js
safety/\${routePath}			
GET	/api/safety/\${routePath}	Login	safety-api-supabase.js
POST	/api/safety/\${routePath}	Login	safety-api-supabase.js
DELETE	/api/safety/\${routePath}/:id	Login	safety-api-supabase.js
PUT	/api/safety/\${routePath}/:id	Login	safety-api-supabase.js
safety/agent			

Method	Path	Access	Source
GET	/api/safety/agent/proposals	Login	safety-agent-api-supabase.js
GET	/api/safety/agent/proposals/:id	Login	safety-agent-api-supabase.js
POST	/api/safety/agent/proposals/:id/approve	Login	safety-agent-api-supabase.js
POST	/api/safety/agent/proposals/:id/reject	Login	safety-agent-api-supabase.js
POST	/api/safety/agent/propose	Login	safety-agent-api-supabase.js
POST	/api/safety/agent/upload	Login	safety-agent-api-supabase.js
safety/audits			
GET	/api/safety/audits	Login	safety-api-supabase.js
POST	/api/safety/audits	Login	safety-api-supabase.js
POST	/api/safety/audits/:auditId/items/:itemId/corrective-actions	Login	safety-api-supabase.js
GET	/api/safety/audits/:id	Login	safety-api-supabase.js
POST	/api/safety/audits/:id/items	Login	safety-api-supabase.js
PUT	/api/safety/audits/:id/status	Login	safety-api-supabase.js
safety/checklist			
POST	/api/safety/checklist/headings	Login	routes/checklist-api-supabase.js
DELETE	/api/safety/checklist/headings/:id	Login	routes/checklist-api-supabase.js
GET	/api/safety/checklist/headings/:tabId	Login	routes/checklist-api-supabase.js
POST	/api/safety/checklist/init	Login	routes/checklist-api-supabase.js
POST	/api/safety/checklist/items	Login	routes/checklist-api-supabase.js
DELETE	/api/safety/checklist/items/:id	Login	routes/checklist-api-supabase.js
GET	/api/safety/checklist/items/:tabId	Login	routes/checklist-api-supabase.js
POST	/api/safety/checklist/search	Login	routes/checklist-suggestions.js
POST	/api/safety/checklist/suggestions	Login	routes/checklist-suggestions.js
safety/grievances			
GET	/api/safety/grievances	Login	safety-api-supabase.js
POST	/api/safety/grievances	Login	safety-api-supabase.js
DELETE	/api/safety/grievances/:id	Login	safety-api-supabase.js
PUT	/api/safety/grievances/:id	Login	safety-api-supabase.js
safety/incidents			
GET	/api/safety/incidents	Login	safety-api-supabase.js
POST	/api/safety/incidents	Login	safety-api-supabase.js
DELETE	/api/safety/incidents/:id	Login	safety-api-supabase.js
GET	/api/safety/incidents/:id	Login	safety-api-supabase.js
PUT	/api/safety/incidents/:id	Login	safety-api-supabase.js
safety/quotes			

Method	Path	Access	Source
PUT	/api/safety/quotes/:id/approve	Login	safety-api-supabase.js
GET	/api/safety/quotes/:id/invoice	Login	safety-api-supabase.js
POST	/api/safety/quotes/:id/invoice	Login	safety-api-supabase.js
PUT	/api/safety/quotes/:id/status	Login	safety-api-supabase.js
safety/rfq			
GET	/api/safety/rfq	Login	safety-api-supabase.js
POST	/api/safety/rfq	Login	safety-api-supabase.js
DELETE	/api/safety/rfq/:id	Login	safety-api-supabase.js
PUT	/api/safety/rfq/:id	Login	safety-api-supabase.js
GET	/api/safety/rfq/:id/quotes	Login	safety-api-supabase.js
POST	/api/safety/rfq/:id/quotes	Login	safety-api-supabase.js
GET	/api/safety/rfq/by-item/:auditItemId	Login	safety-api-supabase.js
GET	/api/safety/rfq/quote-stats	Login	safety-api-supabase.js
safety/self			
GET	/api/safety/self-check	Public	safety-api-supabase.js
safety/training			
GET	/api/safety/training-sessions	Login	safety-api-supabase.js
POST	/api/safety/training-sessions	Login	safety-api-supabase.js
DELETE	/api/safety/training-sessions/:id	Login	safety-api-supabase.js
GET	/api/safety/training-sessions/:id	Login	safety-api-supabase.js
PUT	/api/safety/training-sessions/:id	Login	safety-api-supabase.js
POST	/api/safety/training-sessions/:id/attendees	Login	safety-api-supabase.js
safety/units			
GET	/api/safety/units	Login	safety-api-supabase.js
POST	/api/safety/units	Login	safety-api-supabase.js
DELETE	/api/safety/units/:id	Login	safety-api-supabase.js
support			
GET	/api/support/tickets	Login	web-server.js
POST	/api/support/tickets	Login	web-server.js
POST	/api/support/tickets/:id/messages	Login	web-server.js
tasks			
GET	/api/tasks	Login	web-server.js
POST	/api/tasks	Login	web-server.js
DELETE	/api/tasks/:id	Login	web-server.js
PUT	/api/tasks/:id	Login	web-server.js
tenant			

Method	Path	Access	Source
POST	/api/tenant/:tenantId/assign-role	Login	web-server.js
GET	/api/tenant/:tenantId/employees	Login	web-server.js
POST	/api/tenant/:tenantId/employees	Login	web-server.js
GET	/api/tenant/:tenantId/roles	Login	web-server.js
POST	/api/tenant/:tenantId/roles	Login	web-server.js
GET	/api/tenant/:tenantId/summary	Login	web-server.js
tenants			
GET	/api/tenants/by-email	Login	web-server.js
test-connection			
POST	/api/test-connection	Public	web-server.js
uploads			
POST	/api/uploads	Login	web-server.js
waste			
GET	/api/waste/management	Login	routes/waste-management-supabase.js
POST	/api/waste/management	Login	routes/waste-management-supabase.js
DELETE	/api/waste/management/:id	Login	routes/waste-management-supabase.js
GET	/api/waste/management/:id	Login	routes/waste-management-supabase.js
PUT	/api/waste/management/:id	Login	routes/waste-management-supabase.js
water			
GET	/api/water/buying	Login	routes/water-management-supabase.js
POST	/api/water/buying	Login	routes/water-management-supabase.js
DELETE	/api/water/buying/:id	Login	routes/water-management-supabase.js
GET	/api/water/buying/:id	Login	routes/water-management-supabase.js
PUT	/api/water/buying/:id	Login	routes/water-management-supabase.js
GET	/api/water/discharge	Login	routes/water-management-supabase.js
POST	/api/water/discharge	Login	routes/water-management-supabase.js
DELETE	/api/water/discharge/:id	Login	routes/water-management-supabase.js

Method	Path	Access	Source
GET	/api/water/discharge/:id	Login	routes/water-management-supabase.js
PUT	/api/water/discharge/:id	Login	routes/water-management-supabase.js
GET	/api/water/rain	Login	routes/water-management-supabase.js
POST	/api/water/rain	Login	routes/water-management-supabase.js
DELETE	/api/water/rain/:id	Login	routes/water-management-supabase.js
GET	/api/water/rain/:id	Login	routes/water-management-supabase.js
PUT	/api/water/rain/:id	Login	routes/water-management-supabase.js
GET	/api/water/recycling	Login	routes/water-management-supabase.js
POST	/api/water/recycling	Login	routes/water-management-supabase.js
DELETE	/api/water/recycling/:id	Login	routes/water-management-supabase.js
GET	/api/water/recycling/:id	Login	routes/water-management-supabase.js
PUT	/api/water/recycling/:id	Login	routes/water-management-supabase.js
GET	/api/water/usage	Login	routes/water-management-supabase.js
POST	/api/water/usage	Login	routes/water-management-supabase.js
DELETE	/api/water/usage/:id	Login	routes/water-management-supabase.js
GET	/api/water/usage/:id	Login	routes/water-management-supabase.js
PUT	/api/water/usage/:id	Login	routes/water-management-supabase.js
GET	/api/water/waste	Login	routes/water-management-supabase.js
POST	/api/water/waste	Login	routes/water-management-supabase.js
DELETE	/api/water/waste/:id	Login	routes/water-management-supabase.js
GET	/api/water/waste/:id	Login	routes/water-management-supabase.js
PUT	/api/water/waste/:id	Login	routes/water-management-supabase.js

8. Security

8.1 Controls in place

- Passwords hashed with scrypt and a per-user random salt; optional TOTP/email OTP; optional reCAPTCHA.
- HttpOnly, SameSite=Lax, Secure (production) session cookie with 7-day lifetime.
- Tenant isolation enforced server-side for safety, water, waste, attendance and AI-prompt data; client-supplied tenant ids are ignored; cross-tenant record ids return 404. Verified by an automated two-tenant isolation test (30 checks).
- Payroll gated by HR verification and per-period confirmation; tax and attendance administration restricted by permission.
- Attendance decisions made server-side only; one-time challenges prevent replay; server clock is authoritative.
- Same-site-only `next` redirects (no open redirect).
- Secrets kept in environment variables; `.env*` files git-ignored.

8.2 Known gaps and recommendations

#	Finding	Recommendation
0	Fixed: <code>POST /api/auth/manual-verify</code> and <code>POST /api/auth/test-smtp</code> previously required no login (anyone could mark accounts verified or send mail through the SMTP account).	Now require a signed-in user with <code>USER_MANAGE</code> ; manual verification is limited to users of the admin's own company.
1	Fixed: endpoints without a login check — tasks (shared across companies), <code>/api/tenants/by-email</code> (company lookup for any email), <code>/api/demo/seed-employees</code> , and <code>/api/agent/status</code> (exposed server/database names).	Tasks are login + company scoped; by-email only for your own email; demo seeding admin-only; status returns no infrastructure names. Remaining public endpoints are intentional (login, register, health, self-check, Stripe webhook).
2	<code>cors()</code> allows any origin.	Restrict to the application's own domain(s).
3	Other modules (CRM, HR, accounting) rely on <code>req.auth.tid</code> per route; they were not part of the isolation test.	Extend the two-tenant test to every module.
4	Fixed: generating payroll twice for a month created duplicate AP invoices.	One run per company per month (<code>hr_payroll_run</code>).
5	Demo accounts with public passwords work while <code>AUTO_SEED_DEMO</code> is not <code>0</code> .	Set <code>AUTO_SEED_DEMO=0</code> in production: demo buttons are hidden and demo logins refused.
6	Face liveness is not certified anti-spoofing; browsers cannot detect mock-GPS apps.	Combine with the fingerprint scanner (planned) and HR review.
7	Attachments are stored as base64 data URLs in database rows.	Move files to object storage (e.g. S3-compatible) for size and performance.

8	No automated test suite in the repository (<code>npm test</code> is a stub).	Commit the smoke, isolation, audit, tax and attendance scripts as tests and run them in CI.
---	---	---

9. Deployment and operations

9.1 Railway

- **Build:** Nixpacks, `npm install` (no build step). The face libraries add ~330 MB to `node_modules`.
- **Start:** `node src/web-server.js`; health check `/health` (timeout 300 s); restart on failure (3 retries).
- **Deploy:** push to `main` on GitHub (`sourcingease/Safety`); Railway builds and deploys automatically.
- **Variables:** Safety service → Variables (Raw Editor). Database: the Postgres service; the app references `{{Postgres.DATABASE_URL}}`.

9.2 Database changes

1. Write an idempotent SQL file `supabase/NN_description.sql` and add it to `scripts/setup-supabase.js`.
2. Apply it to the Railway database (public URL) **before** deploying code that depends on it, or make it backward compatible (nullable columns first).
3. For changes that the running code cannot tolerate (e.g. new NOT NULL columns), apply and deploy back-to-back.

9.3 Local development

```
git clone https://github.com/sourcingease/Safety.git
cd Safety && npm install
# .env: SUPABASE_DB_URL=<Railway public URL>, ENABLE_SUPABASE_*=1, JWT_SECRET, PORT=3003
node src/web-server.js      # http://localhost:3003
npm run supabase:setup      # apply all SQL files to an empty database
```

9.4 Monitoring and maintenance

- `GET /health` returns status and uptime. `GET /api/safety/self-check` checks database connectivity.
- Logs: Railway → Deployments → Logs. Expected start-up lines include "Postgres safety routes initialized" and "Face engine ready".
- Back up the Postgres service (Railway backups or `pg_dump` with a client $\geq$ v18).
- Rotate the database password if it is ever exposed; update the local `.env` (the app uses the variable reference automatically).

10. Known limitations and roadmap

Area	Status
Buyer, Supplier, Designer, Inspection, Manufacturer portals	Placeholder pages (title and quick links only)
Reports, Tracking, CRM Catalog/Inventory/Webinars	Placeholder layouts, not connected to data
Company switching	API exists (/api/auth/switch-tenant); no UI
Attendance phases 3–4	Planned: phone binding (WebAuthn) and fingerprint-scanner integration (ZKTeco ADMS push)
Payroll absences	Absent-day deduction is a placeholder (0); attendance data is not yet used by payroll
Verification email	Depends on a correctly configured SMTP sender domain